3. V8的垃圾回收与数据存储

在上一节中，我们从**变量提升**引入，讲到了包含变量环境的**执行上下文**，通过执行上下文的栈存储机制，了解了什么是**调用栈**。

接着我们提到了变量提升的缺点，引入let和const，他们通过**块级作用域**的机制，在执行上文文中，占据了词法环境的一席之地。

接着，我们在嵌套函数调用中，讲解了**作用域链**，当我们查找变量时，在当前执行上下文无法查到时，是如何通过作用域链继续往外查找。

作用域链并不是根据调用函数的顺序链起来的，而是通过**词法作用域**，词法作用域是静态的，在我们编写代码时就已经确定，所以作用域链也是静态的。

最后，我们从作用域链讲解了**闭包**的概念。这就是上一节课的主要内容。

接下来进入我们这一节课：

在开发人员中，有两个我们常常提及的词语，分别是内存泄漏与垃圾回收，前者会导致卡顿问题，也是面试官常问的问题，后者则帮助我们更好的理解内存泄漏。

为了搞清**JavaScript 的内存机制**，在接下来的三节（数据在内存中的存放、JavaScript 处理垃圾回收以及 V8 执行代码）中，我们将通过内存机制的介绍，循序渐进带你走进 JavaScript 内存的世界。

数据在内存中的存放

在先说数据之前，我们得先从“JavaScript 是什么类型的语言”讲起。我们把这种在使用之前就需要确认其变量数据类型的称为**静态语言**。相反地，我们把在运行过程中需要检查数据类型的语言称为**动态语言**。比如我们所讲的 JavaScript 就是动态语言，因为在声明变量之前并不需要确认其数据类型。让我们再来认识一下静态语言的写法，比如C：

C++

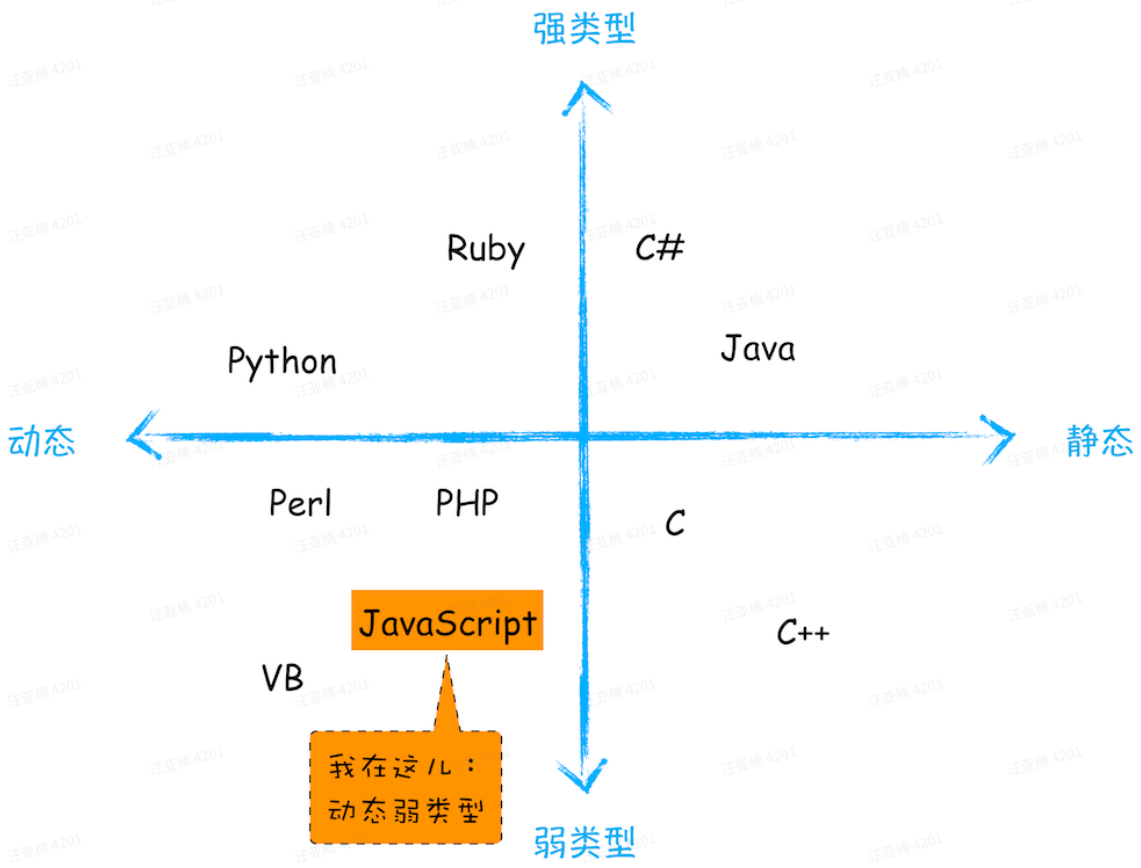
```
int main()
{
    int a = 1;
    char* b = "IDD";
    bool c = true;
    return 0;
}
```

可以看到C语言的声明变量特色：在声明变量之前需要先定义变量类型。虽然C语言是静态语言，但是在 C 语言中，我们可以把其他类型数据赋予给一个声明好的变量，如：

```
C++
c = a
```

前面代码中，我们把 int 型的变量 a 赋值给了 bool 型的变量 c，这段代码也是可以编译执行的，因为在赋值过程中，C 编译器会把 int 型的变量悄悄转换为 bool 型的变量，我们通常把这种偷偷转换的操作称为**隐式类型转换**。而支持隐式类型转换的语言称为**弱类型语言**，不支持隐式类型转换的语言称为**强类型语言**。在这点上，C 和 JavaScript 都是弱类型语言。

对于各种语言的类型，可以参考下图



JavaScript 的数据类型

现在我们知道了，JavaScript 是一种弱类型的、动态的语言。那这些特点意味着什么呢？

- 弱类型，意味着你不需要告诉 JavaScript 引擎这个或那个变量是什么数据类型，JavaScript 引擎在运行代码的时候自己会计算出来。
- 动态，意味着你可以使用同一个变量保存不同类型的数据。

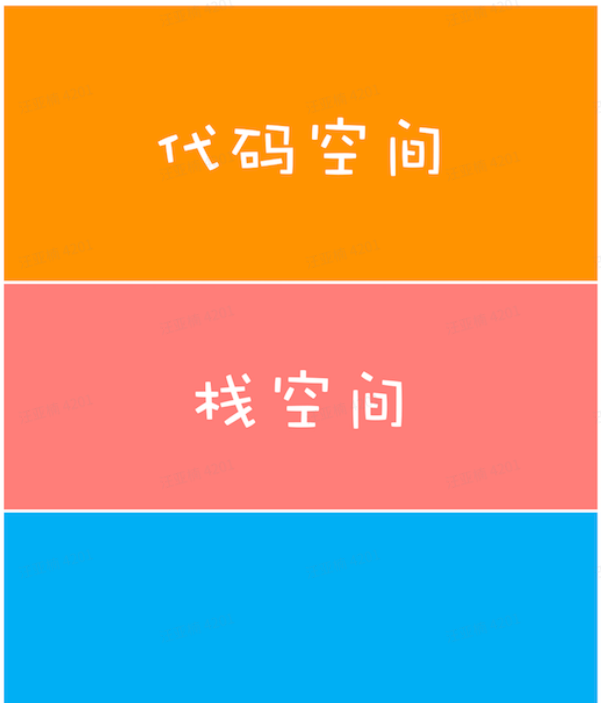
其实 JavaScript 中的数据类型一种有 8 种，它们分别是

类型	描述
Boolean	只有true和false两个值。
Null	只有一个值null。
Undefined	一个没有被赋值的变量会有个默认值 undefined，变量提升时的默认值也是undefined。
Number	根据 ECMAScript 标准，JavaScript 中只有一种数字类型：基于 IEEE 754 标准的双精度 64 位二进制格式的值，-(263 -1) 到 263 -1。
BigInt	JavaScript 中一个新的数字类型，可以用任意精度表示整数。使用 BigInt，即使超出 Number 的安全整数范围限制，也可以安全地存储和操作。
String	用于表示文本数据。不同于类 C 语言，JavaScript 的字符串是不可更改的。
Symbol	符号类型是唯一的并且是不可修改的，通常用来作为Object的key。
Object	在 JavaScript 里，对象可以被看作是一组属性的集合。

我们把前面的 7 种数据类型称为**原始类型**，把最后一个对象类型称为**引用类型**，之所以把它们区分为两种不同的类型，是因为它们在内存中存放的位置不一样。到底怎么个不一样法呢？接下来，我们就来讲解一下 JavaScript 的原始类型和引用类型到底是怎么储存的。

内存空间

要理解 JavaScript 在运行过程中数据是如何存储的，你就得先搞清楚其存储空间的种类。以下是 JavaScript 的内存模型，可以大概参考下：



从图中可以看出， 在 JavaScript 的执行过程中， 主要有三种类型内存空间，分别是代码空间、栈空间和堆空间。今天我们主要来说说栈空间和堆空间。

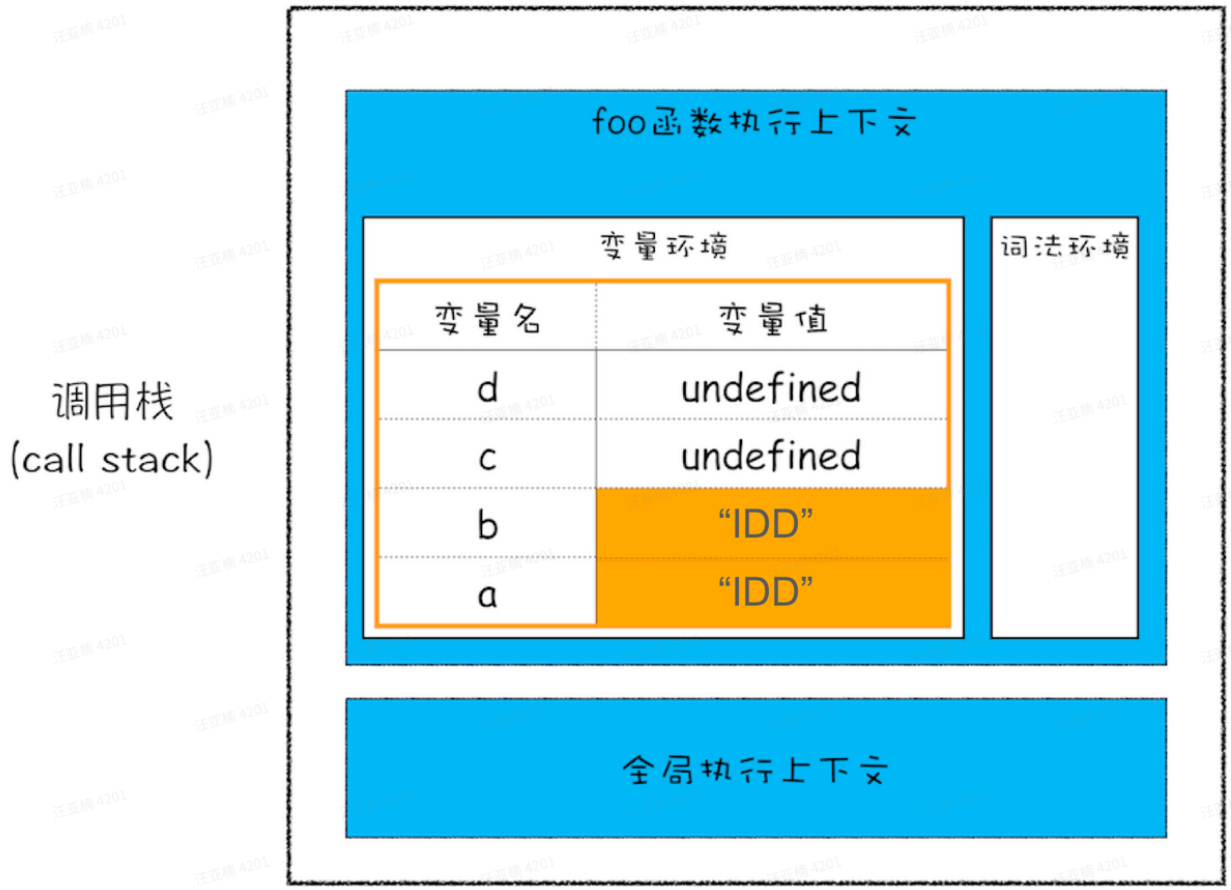
栈空间和堆空间

这里的栈空间就是我们之前反复提及的调用栈，是用来存储执行上下文的。为了搞清楚栈空间是如何存储数据的，我们还是先看下面这段代码：

JavaScript

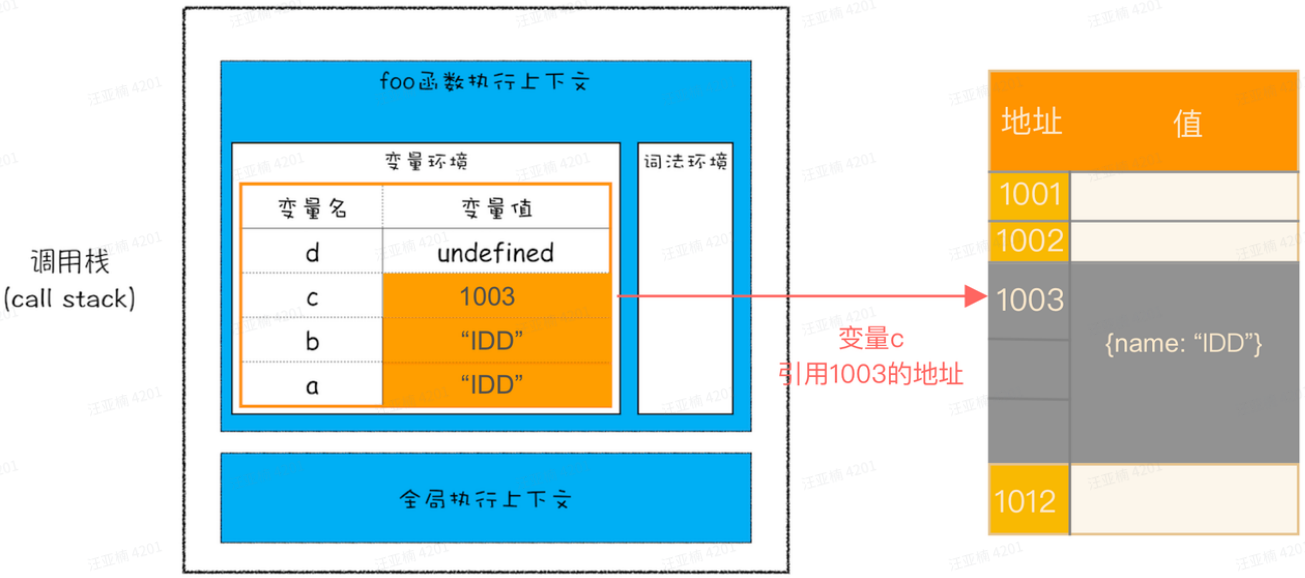
```javascript
1  function foo(){
2      var a = "IDD"
3      var b = a
4      var c = {name:"IDD"}
5      var d = c
6  }
7  foo()
```

前面文章我们已经讲解过了，当执行一段代码时，需要先编译，并创建执行上下文，然后再按照顺序执行代码。那么下面我们来看看，当执行到第 3 行代码时，其调用栈的状态，你可以参考下面这张调用栈状态图：



从图中可以看出来，当执行到第 3 行时，变量 a 和变量 b 的值都被保存在执行上下文中，而执行上下文又被压入到栈中，所以你也可以认为变量 a 和变量 b 的值都是存放在栈中的。

接下来继续执行第 4 行代码，由于 JavaScript 引擎判断右边的值是一个引用类型，这时候处理的情况就不一样了，JavaScript 引擎并不是直接将该对象存放到变量环境中，而是将它分配到堆空间里面，分配后该对象会有一个在“堆”中的地址，然后再将该数据的地址写进 c 的变量值，最终分配好内存的示意图如下所示：



从上图你可以清晰地观察到，对象类型是存放在堆空间的，在栈空间中只是保留了对象的引用地址，当 JavaScript 需要访问该数据的时候，是通过栈中的引用地址来访问的，相当于多了一道转手流程。

好了，现在你应该知道了原始类型的数据值都是直接保存在“栈”中的，引用类型的值是存放在“堆”中的。不过你也许会好奇，为什么一定要分“堆”和“栈”两个存储空间呢？所有数据直接存放在“栈”中不就可以了吗？

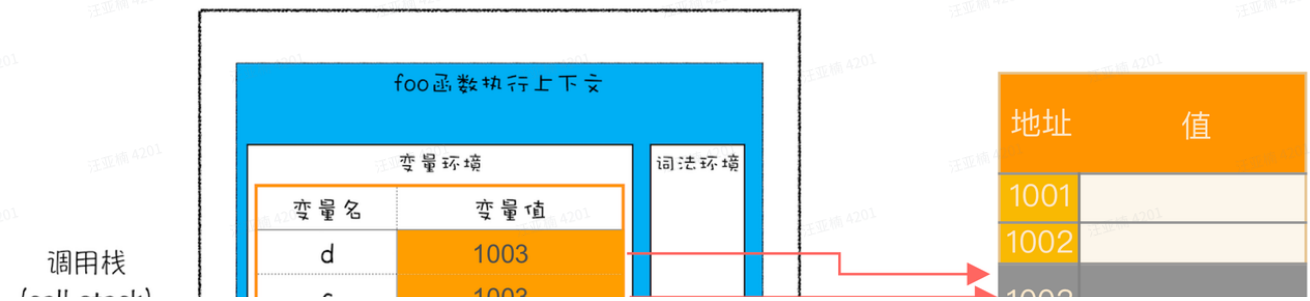
答案是不可以的。这是因为 JavaScript 引擎需要用栈来维护程序执行期间上下文的状态，如果栈空间大了话，所有的数据都存放在栈空间里面，那么会影响到上下文切换的效率，进而又影响到整个程序的执行效率。

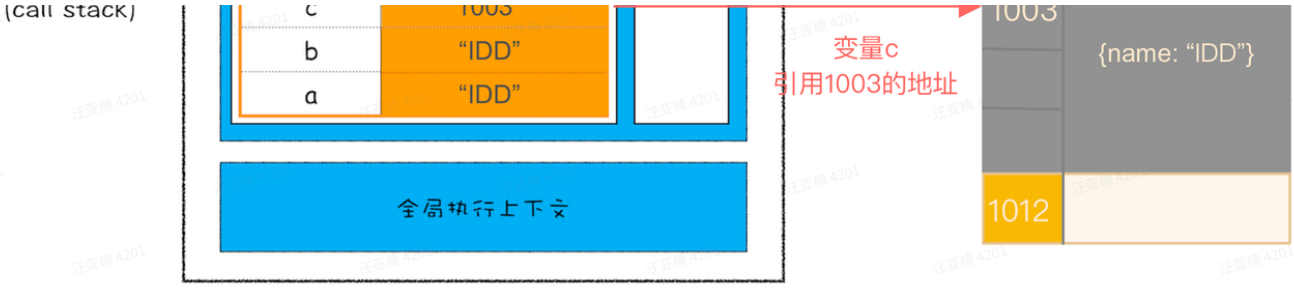
所以通常情况下，栈空间都不会设置太大，主要用来存放一些原始类型的小数据。而引用类型的数据占用的空间都比较大，所以这一类数据会被存放到堆中，堆空间很大，能存放很多大的数据，不过缺点是分配内存和回收内存都会占用一定的时间。

解释了程序在执行过程中为什么需要堆和栈两种数据结构后，我们还是回到示例代码那里，看看它最后一步将变量 c 赋值给变量 d 是怎么执行的？

在 JavaScript 中，赋值操作和其他语言有很大的不同，原始类型的赋值会完整复制变量值，而引用类型的赋值是复制引用地址。

所以d=c的操作就是把 c 的引用地址赋值给 d，你可以参考下图：





从图中你可以看到，变量 c 和变量 d 都指向了同一个堆中的对象，所以这就很好地解释了文章开头的那个问题，通过 c 修改 name 的值，变量 d 的值也跟着改变，归根结底它们是同一个对象。

当执行这段代码的时候，你应该有过这样的分析：由于变量 myName、test1、test2 都是原始类型数据，所以在执行 foo 函数的时候，它们会被压入到调用栈中；当 foo 函数执行结束之后，调用栈中 foo 函数的执行上下文会被销毁，其内部变量 myName、test1、test2 也应该一同被销毁。

再谈闭包

现在你知道了作用域内的原始类型数据会被存储到栈空间，引用类型会被存储到堆空间，基于这两点的认知，我们再深入一步，探讨下闭包的内存模型。

JavaScript

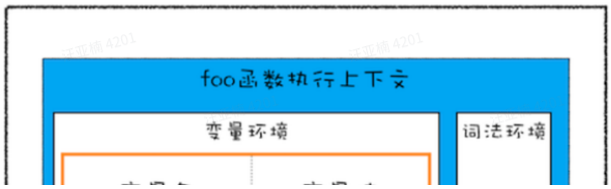
```javascript
function foo() {
    var myName = "IDD"
    let test1 = 1
    const test2 = 2
    var innerBar = {
        setName:function(newName){
            myName = newName
        },
        getName:function(){
            console.log(test1)
            return myName
        }
    }
    return innerBar
}
var bar = foo()
bar.setName("前段组")
bar.getName()
console.log(bar.getName())
```

我们介绍了当 foo 函数的执行上下文销毁时，由于 foo 函数产生了闭包，所以变量 myName 和 test1 并没有被销毁，而是保存在内存中，那么应该如何解释这个现象呢？

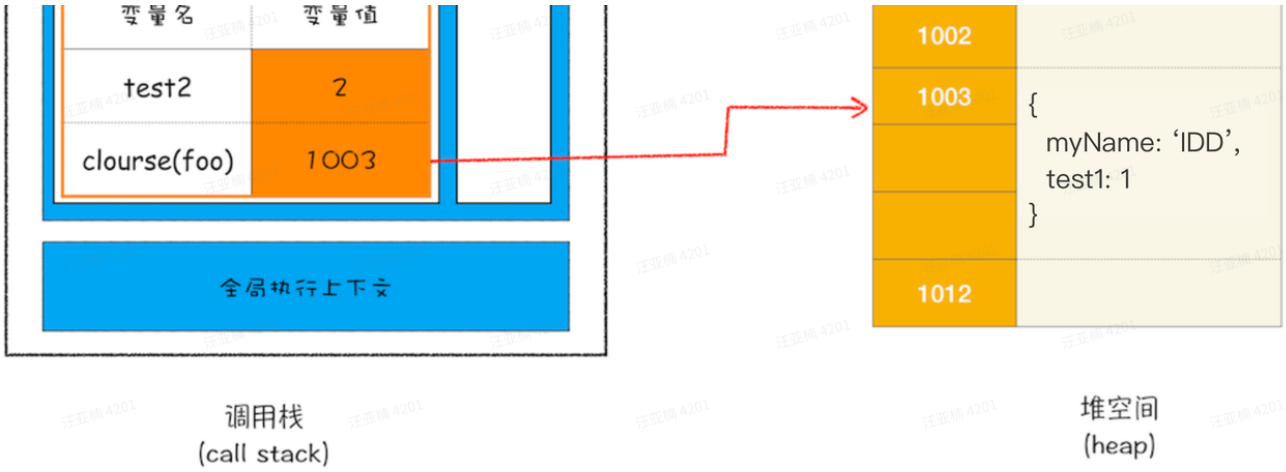
要解释这个现象，我们就得站在内存模型的角度来分析这段代码的执行流程。

1. 当 JavaScript 引擎执行到 foo 函数时，首先会编译，并创建一个空执行上下文。
2. 在编译过程中，遇到内部函数 setName，JavaScript 引擎还要对内部函数做一次快速的词法扫描，发现该内部函数引用了 foo 函数中的 myName 变量，由于是内部函数引用了外部函数的变量，所以 JavaScript 引擎判断这是一个闭包，于是在堆空间创建换一个“closure(foo)”的对象（这是一个内部对象，JavaScript 是无法访问的），用来保存 myName 变量。
3. 接着继续扫描到 getName 方法时，发现该函数内部还引用变量 test1，于是 JavaScript 引擎又将 test1 添加到“closure(foo)”对象中。这时候堆中的“closure(foo)”对象中就包含了 myName 和 test1 两个变量了。
4. 由于 test2 并没有被内部函数引用，所以 test2 依然保存在调用栈中。

通过上面的分析，我们可以画出执行到 foo 函数中“return innerBar”语句时的调用栈状态，如下图所示：



地址	值
1001	



从上图你可以清晰地看出，当执行到 foo 函数时，闭包就产生了；当 foo 函数执行结束之后，返回的 getName 和 setName 方法都引用“closure(foo)”对象，所以即使 foo 函数退出了，“closure(foo)”依然被其内部的 getName 和 setName 方法引用。所以在下次调用bar.setName或者bar.getName时，创建的执行上下文中就包含了“closure(foo)”。

总的来说，产生闭包的核心有两步：第一步是需要预扫描内部函数；第二步是把内部函数引用的外部变量保存到堆中。

垃圾数据是如何自动回收的

上一小节中，我们提到了 JavaScript 中的数据是如何存储的，并通过例子分析了原始数据类型是存储在栈空间中的，引用类型的数据是存储在堆空间中的。通过这种分配方式，我们解决了数据的内存分配的问题。

不过有些数据被使用之后，可能就不再需要了，我们把这种数据称为**垃圾数据**。如果这些垃圾数据一直保存在内存中，那么内存会越用越多，所以我们需要**对这些垃圾数据进行回收，以释放有限的内存空间。**

通常情况下，垃圾数据回收分为**手动回收**和**自动回收**两种策略。

如 C/C++ 就是使用手动回收策略，何时分配内存、何时销毁内存都是由代码控制的，当不再需要数据的时候，就要手动调用销毁函数来释放内存。如果这段数据已经不再需要了，但是又没有主动调用销毁函数来销毁，那么这种情况就被称为**内存泄漏**。

另外一种使用的是自动垃圾回收的策略，如 JavaScript、Java、Python 等语言，产生的垃圾数据是由垃圾回收器来释放的，并不需要手动通过代码来释放。

但误以为可以不关心内存管理，这是一个很大的误解。接下来我们将探讨垃圾回收的机制，因为数据是存储在栈和堆两种内存空间中的，所以接下来我们就来分别介绍“栈中的垃圾数据”和“堆中的垃

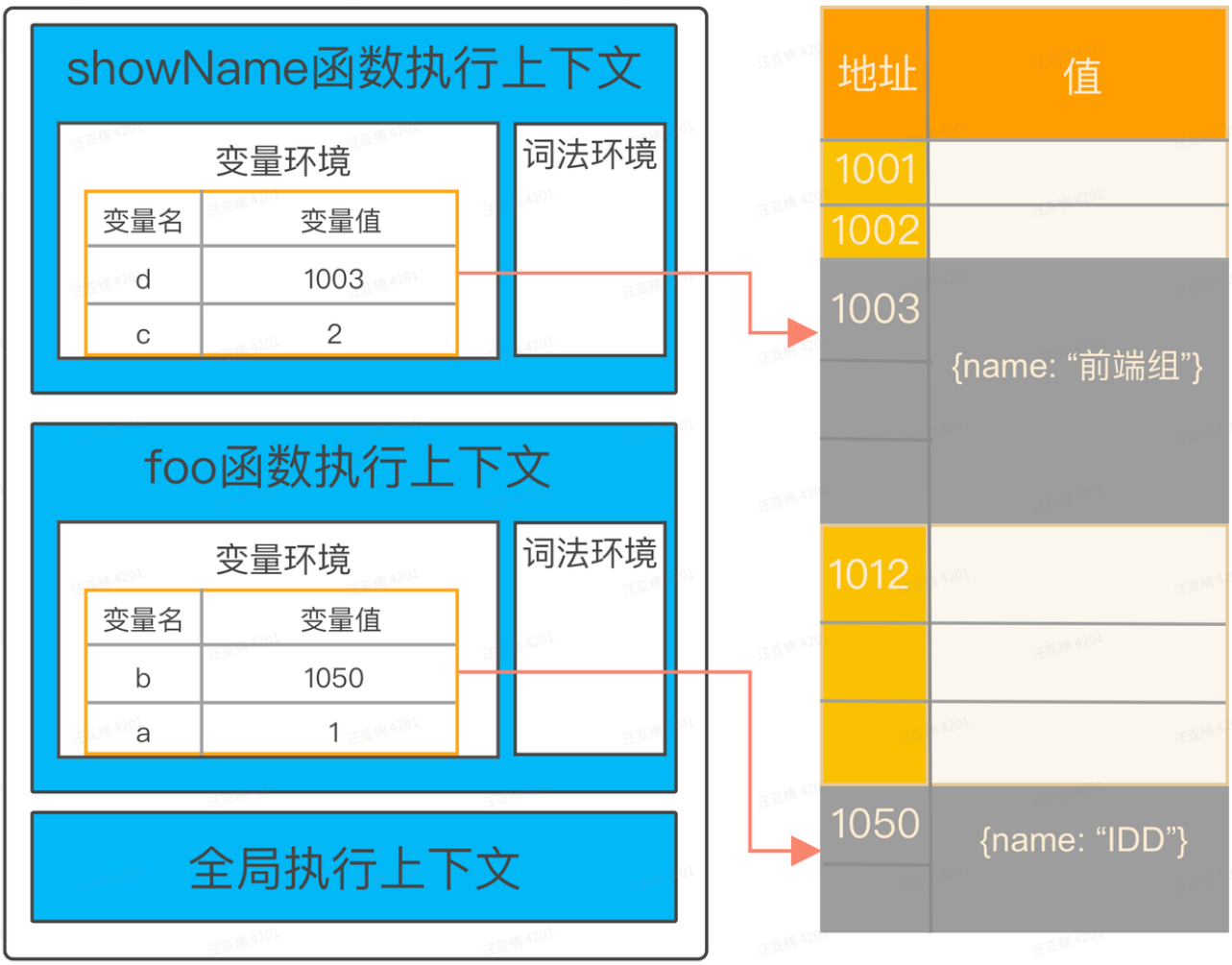
圾数据”是如何回收的。

调用栈中的垃圾是如何回收的

JavaScript

```
1  function foo(){
2      var a = 1
3      var b = {name:"IDD"}
4      function showName(){
5        var c = 2
6        var d = {name:"前端组"}
7      }
8      showName()
9  }
10 foo()
```

当执行到第 6 行代码时，其调用栈和堆空间状态图如下所示：



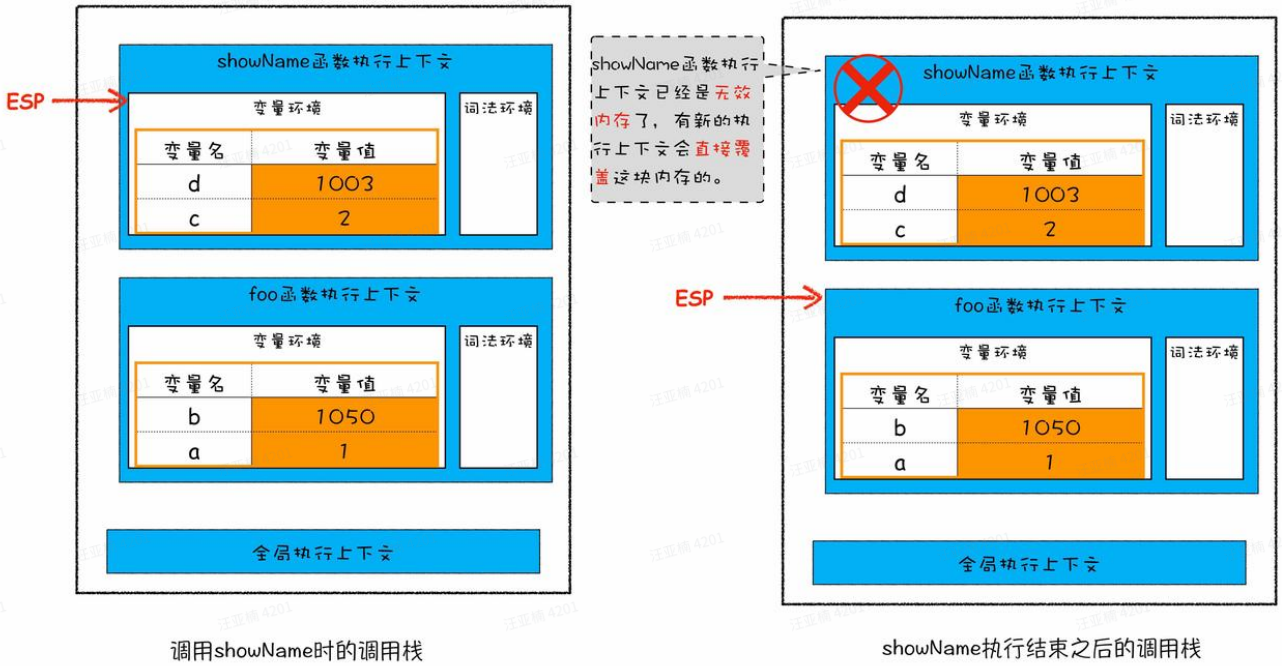
从图中可以看出，原始类型的数据被分配到栈中，引用类型的数据会被分配到堆中。当 foo 函数执行结束之后，foo 函数的执行上下文会从堆中被销毁掉，那么它是怎么被销毁的呢？下面我们就来分析

一下。

我们简单介绍过了，如果执行到 showName 函数时，那么 JavaScript 引擎会创建 showName 函数的执行上下文，并将 showName 函数的执行上下文压入到调用栈中，最终执行到 showName 函数时，其调用栈就如上图所示。与此同时，还有一个**记录当前执行状态的指针（称为 ESP）**，指向调用栈中 showName 函数的执行上下文，表示当前正在执行 showName 函数。

接着，当 showName 函数执行完成之后，函数执行流程就进入了 foo 函数，那这时就需要销毁 showName 函数的执行上下文了。ESP 这时候就帮上忙了，JavaScript 会将 ESP 下移到 foo 函数的执行上下文，**这个下移操作就是销毁 showName 函数执行上下文的过程。**

你可能会有点懵，ESP 指针向下移动怎么就能把 showName 的执行上下文销毁了呢？具体你可以看下面这张移动 ESP 前后的对比图：



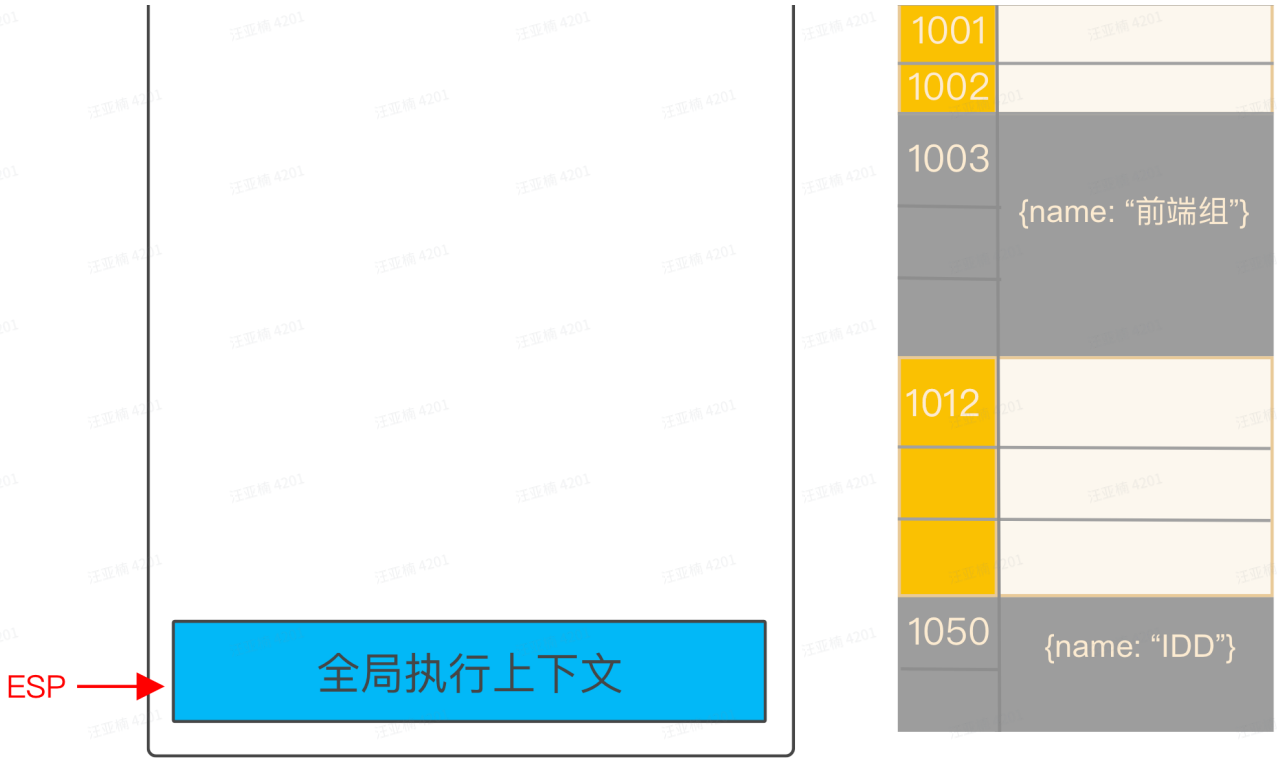
调用showName时的调用栈　　showName执行结束之后的调用栈

从图中可以看出，当 showName 函数执行结束之后，ESP 向下移动到 foo 函数的执行上下文中，上面 showName 的执行上下文虽然保存在栈内存中，但是已经是无效内存了。比如当 foo 函数再次调用另外一个函数时，这块内容会被直接覆盖掉，用来存放另外一个函数的执行上下文。

所以说，当一个函数执行结束之后，**JavaScript 引擎会通过向下移动 ESP 来销毁该函数保存在栈中的执行上下文。**

堆中的数据是如何回收的

通过上面的讲解，我想现在你应该已经知道，当上面那段代码的 foo 函数执行结束之后，ESP 应该是指向全局执行上下文的，那这样的话，showName 函数和 foo 函数的执行上下文就处于无效状态了，不过保存在堆中的两个对象依然占用着空间，如下图所示：



从图中可以看出，1003 和 1050 这两块内存依然被占用。**要回收堆中的垃圾数据，就需要用到 JavaScript 中的垃圾回收器了。**

所以，接下来我们就来通过 Chrome 的 JavaScript 引擎 V8 来分析下堆中的垃圾数据是如何回收的。

代际假说和分代收集

不过在正式介绍 V8 是如何实现回收之前，你需要先学习下**代际假说（The Generational Hypothesis）**的内容，这是垃圾回收领域中一个重要的术语，后续垃圾回收的策略都是建立在该假说的基础之上的，所以很是重要。

代际假说有以下两个特点：

- 第一个是大部分对象在内存中存在的时间很短，简单来说，就是很多对象一经分配内存，很快就变得不可访问；
- 第二个是不死的对象，会活得更久。

其实这两个特点不仅仅适用于 JavaScript，同样适用于大多数的动态语言，如 Java、Python 等。

有了代际假说的基础，我们就可以来探讨 V8 是如何实现垃圾回收的了。

通常，垃圾回收算法有很多种，但是并没有哪一种能胜任所有的场景，你需要权衡各种场景，根据对象的生存周期的不同而使用不同的算法，以便达到最好的效果。

所以，在 V8 中会把堆分为**新生代**和**老生代**两个区域，**新生代中存放的是生存时间短的对象，老生代中存放的生存时间久的对象。**

新生区通常只支持 1～8M 的容量，而老生区支持的容量就大很多了。对于这两块区域，V8 分别使用两个不同的垃圾回收器，以便更高效地实施垃圾回收。

垃圾回收的工作流程

现在你知道了 V8 把堆分成两个区域——新生代和老生代，并分别使用两个不同的垃圾回收器。**其实不论什么类型的垃圾回收器，它们都有一套共同的执行流程。**

1. 标记空间中活动对象和非活动对象。所谓活动对象就是还在使用的对象，非活动对象就是可以进行垃圾回收的对象。
2. 回收非活动对象所占据的内存。其实就是在所有的标记完成之后，统一清理内存中所有被标记为可回收的对象。
3. 做内存整理。一般来说，频繁回收对象后，内存中就会存在大量不连续空间，我们把这些不连续的内存空间称为内存碎片。当内存中出现了大量的内存碎片之后，如果需要分配较大连续内存的时候，就有可能出现内存不足的情况。

好了，以上我们讲解了数据存储以及垃圾回收。数据存储包括了栈和堆的存储，而垃圾回收也分为栈和堆两种不同的垃圾回收。

思考时间

在实际的项目中，经常需要完整地拷贝一个对象，也就是说拷贝完成之后两个对象之间就不能互相影响。那该如何实现呢？

结合下面这段代码，你可以分析下它是如何将对象 jack 拷贝给 jack2，然后在完成拷贝操作时两个 jack 还互不影响的呢。

JavaScript

```javascript
let jack = {
    name : "jack.ma",
    age:40,
  like:{
        dog:{
            color:'black',
            age:3,
        },
        cat:{
            color:'white',
            age:2
        }
    }
}
function copy(src){
    let dest
    //实现拷贝代码，将src的值完整地拷贝给dest
  //在这里实现
    return dest
}
let jack2 = copy(jack)

//比如修改jack2中的内容，不会影响到jack中的值
jack2.like.dog.color = 'green'
console.log(jack.like.dog.color) //打印出来的应该是 "black"
```

第二个问题：分享下你是如何判断 JavaScript 中内存泄漏的？可以结合一些你在工作中避免内存泄漏的方法。